

Kubernetes Cluster Troubleshooting Report

END-TO-END CORE INFRASTRUCTURE INCIDENT RESOLUTION

Environment:	WSL (Ubuntu) / KinD (Kubernetes in Docker)	Status:	RESOLVED
Application:	SkillPulse (Multi-Tier App)	Engineer:	DevOps Automation Support

Executive Summary

This document serves as a comprehensive technical post-mortem report detailing the resolution of various structural infrastructure, containerization, syntax alignment, and networking issues encountered during the deployment lifecycle of the local multi-tier application (SkillPulse). Throughout the debugging process, the environment was successfully stabilized within a local KinD (Kubernetes in Docker) engine, ensuring that all components (Frontend, Backend, and Database) are synchronized and running smoothly under high-availability parameters.

Root Cause Analysis & Chronological Incidents

Incident 1: Control Plane Connection Refused (Port 127.0.0.1:38113)

Error Context: `kubectl get pods -n skillpulse` failed with the following console error: `"The connection to the server 127.0.0.1:38113 was refused - did you specify the right host or port?"`

Root Cause: Diagnostic traces revealed that while the background Docker engine was operational, the target Kubernetes Control Plane Node (`kind-skillpulse-control-plane`) had crashed or stopped unexpectedly. This prevented the client-side API engine from initiating a stable control plane handshake loop.

Resolution Workflow & Commands:

The current cluster context and background container states were verified using:

```
kubectl config get-contexts
docker ps -a
```

To eliminate corrupted node cache states, the broken cluster was purged and a fresh, clean orchestration instance was spun up:

```
kind delete cluster --name kind-skillpulse
kind create cluster --name kind-skillpulse
```

Incident 2: Strict Decoding Syntax Error in Backend Manifest

Error Context: Executing `kubectl apply -f k8s` threw structural schema compilation errors: `"error when creating 'k8s/20-backend.yaml': Deployment in version 'v1' cannot be handled as a Deployment: strict decoding error: unknown field 'labels'"`

Root Cause: A clear YAML specification violation. The `labels` key was misaligned and nested under an invalid root-level block instead of being strictly confined within the standard resource `metadata` hierarchy, triggering a strict schema-decoding failure in the Kubernetes API parser.

Resolution Workflow & Commands:

The structural layout of the backend deployment manifest was dynamically overwritten with verified layout parameters using an inline block execution:

```
cat << 'EOF' > k8s/20-backend.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: skillpulse
  labels:
    app: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: trainwithshubham/skillpulse-backend:latest
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8080
          env:
            - name: DB_HOST
              value: "mysql"
            - name: DB_PORT
              value: "3306"
            - name: PORT
```

```
        value: "8080"
      - name: DB_USER
        value: "root"
      - name: DB_PASSWORD
        value: "root"
    resources:
      requests:
        cpu: 100m
EOF
kubectl apply -f k8s/20-backend.yaml
```

Incident 3: Frontend CrashLoopBackOff - Nginx Host Upstream Mismatch

Error Context: The frontend pod was stuck in a persistent `CrashLoopBackOff` state. Container standard output was inspected using:

```
kubectl logs deployment/frontend -n skillpulse --tail=50
```

Log Stack Output: `nginx: [emerg] host not found in upstream "backend" in /etc/nginx/conf.d/default.conf:14`

Root Cause: The reverse proxy server (Nginx) inside the frontend container is configured with strict start-up upstream checks. Since the core `backend` Network Service registry was missing during the initial mass-apply step, CoreDNS could not resolve the hostname `"backend"`, forcing Nginx to exit with a critical emergency state.

Resolution Workflow & Commands:

An isolated cluster service mapping endpoint was dynamically created to register the backend routing route into CoreDNS:

```
cat << 'EOF' > k8s/backend-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: skillpulse
spec:
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
  selector:
    app: backend
EOF
kubectl apply -f k8s/backend-service.yaml
kubectl apply -f .
```

The network service status was confirmed, and a rolling restart was forced onto the frontend application layer to pick up the fresh DNS tables:

```
kubectl get svc -n skillpulse
kubectl rollout restart deployment frontend -n skillpulse
```

DevOps High-Availability Verification Best Practices

To protect against future cascading environmental rollbacks or state degradation patterns, observe the following architectural guidelines:

1. **Infrastructure Topology Validation Matrix:** Cluster initialization sequences must strictly adhere to downstream dependencies: `Namespace` → `Secrets/ConfigMaps` → `Stateful/Database Deployments` → `Database ClusterIP Services` → `Backend Deployments` → `Backend Internal Services` → `Frontend Web Proxies`.
2. **Active Environmental Monitors:** Keep an active log hook running to intercept state variations before they trigger back-offs:

```
kubectl get pods -n skillpulse -w  
kubectl get events -n skillpulse --sort-by='.metadata.creationTimestamp'
```

Generated automatically by DevOps Automation Core Engine Support Matrix Systems. Confidential Standard Operation Document.